TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TLAXIACO

TÓPICOS AVANZADOS DE PROGRAMACIÓN

Presenta:

Cruz Cuevas Jonathan

González Morales Ximena Leidaly

Montalvo Reyes Karen Irais

Carrera:

Ingeniería en Sistemas Computacionales

Producto:

Diseño de estructuras de Librerías DLL

Docente:

Román Cruz José Alfredo

Tlaxiaco, Oax., a 13 de marzo de 2026

"Educación, Ciencia y Tecnología, Progresos día con día" ®

Contenido

TABLA DE ILUSTRACIONES 3

INTRODUCCION 4

OBJETIVO 4

MATERIAL 5

DESCRIPCION 5

Estructura de la librería 6

1. Clase GesTareas 6

2. Clase GestorTareas 7

CONSOLA 10

..... 10

FUNCIONAMIENTO GENERAL 15

RESULTADOS 16

CONCLUSIONES 16

TABLA DE ILUSTRACIONES

1 GesTareas 6
2 GesTareas - Método Mostrar 7
3 Agregar tarea 7
4 Completar tarea 8
5 Eliminar tarea 8
6 Mostrar todas las tareas 8
7 Mostrar tareas pendientes 9
8 Mostrar tareas completadas 9
9 Buscar tarea 9
10 Contadores 10
Ilustración 11 Librería 10
Ilustración 12 menú de la consola 11
Ilustración 13 Código del menú 11
Ilustración 14 opción 1 13
Ilustración 15 Opciones agregadas 13
Ilustración 16 Opción 2 13
Ilustración 17 opción 3 14
Ilustración 18 opción 4 14
Ilustración 19 Ejecución 15
Ilustración 20 salida de la consola 15

INTRODUCCION

En el desarrollo de aplicaciones es común separar la lógica del programa en componentes reutilizables para mejorar la organización del código y facilitar su mantenimiento. Una forma de lograr esto es mediante el uso de librerías, las cuales contienen clases y métodos que pueden ser utilizados por distintos programas.

En este proyecto se desarrolló una librería para la gestión de tareas, cuyo propósito es almacenar, administrar y manipular información relacionada con actividades o pendientes. Esta librería se integra posteriormente a una aplicación de escritorio, permitiendo separar la lógica del manejo de tareas de la interfaz gráfica del programa.

El uso de esta librería permite mantener el código más estructurado, reutilizable y fácil de mantener, además de facilitar la implementación de funcionalidades como agregar, eliminar, completar o buscar tareas dentro del sistema.

OBJETIVO

Desarrollar una librería en C# que permita gestionar tareas mediante clases y métodos reutilizables para ser utilizados dentro de una aplicación de escritorio.

- Implementar clases que representen las tareas del sistema.
- Crear métodos que permitan agregar, eliminar, completar y buscar tareas.
- Organizar la información mediante listas dinámicas.
- Facilitar la integración de la lógica del sistema con la interfaz gráfica del programa.

MATERIAL

Para el desarrollo de la librería de gestión de tareas se utilizaron los siguientes recursos y herramientas:

1. Software
   - Visual Studio: entorno de desarrollo integrado utilizado para programar la librería y la aplicación de escritorio.
   - Lenguaje de programación C#: utilizado para implementar las clases y métodos que gestionan las tareas.
2. Recursos de programación
   - .NET Framework / .NET: plataforma utilizada para el desarrollo de aplicaciones y librerías en C#.
   - Colecciones dinámicas (List): estructura utilizada para almacenar y administrar las tareas dentro del sistema.
3. Computadora personal para el desarrollo, compilación y prueba del programa.

DESCRIPCION

La librería desarrollada se encarga de administrar todas las tareas registradas dentro del sistema. Para ello se implementaron dos clases principales que trabajan en conjunto para gestionar la información.

La librería permite realizar las siguientes acciones:

- Registrar nuevas tareas.
- Marcar tareas como completadas.
- Eliminar tareas existentes.
- Mostrar todas las tareas registradas.
- Mostrar únicamente tareas pendientes.
- Mostrar tareas completadas.
- Buscar tareas mediante palabras clave.

- Contar el número total de tareas y las que han sido completadas.

Esta funcionalidad permite simular el comportamiento de un sistema de organización de actividades.

Estructura de la librería

La librería está compuesta por dos clases principales:

1. Clase GesTareas

Esta clase representa una tarea individual dentro del sistema. Cada tarea contiene la información necesaria para identificarla y describirla.

Los atributos principales de esta clase son:

- **Id**: identificador único de la tarea.
- **Titulo**: nombre de la tarea.
- **Categoria**: tipo de tarea o grupo al que pertenece.
- **Fecha**: fecha asociada a la tarea.
- **Prioridad**: nivel de importancia de la tarea (baja, media o alta).
- **Completada**: indica si la tarea ya fue finalizada.

```
public class GesTareas
{
    public int Id;
    public string Titulo;
    public string Categoria;
    public string Fecha;
    public int Prioridad;           // 1 = baja, 2 = media, 3 = alta
    public bool Completada;         // true si ya se terminó

    // Constructor
    1 referencia
    public GesTareas(int id, string titulo, string categoria, string fecha, int prioridad)
    {
        Id = id;
        Titulo = titulo;
        Categoria = categoria;
        Fecha = fecha;
        Prioridad = prioridad;
        Completada = false;
    }
```

1 GesTareas

Además, esta clase incluye un método que permite mostrar la información de la tarea en formato de texto, facilitando su visualización dentro del programa.

```
    // metodo mostrar la tarea como texto
    4 referencias
    public string Mostrar()
    {
        string estado;
        if (Completada == true)
            estado = "[✓]";
        else
            estado = "[ ]";

        string prioTexto = "";
        if (Prioridad == 1) prioTexto = "Baja";
        else if (Prioridad == 2) prioTexto = "Media";
        else prioTexto = "Alta";

        string cat = "";
        if (!string.IsNullOrEmpty(Categoria))
            cat = Categoria + " - ";

        return estado + " " + cat + Titulo + "   " + Fecha + "   Prioridad: " + prioTexto;
    }
}
```

2 GesTareas - Método Mostrar

2. Clase GestorTareas

Esta clase es la encargada de administrar todas las tareas del sistema. Para ello utiliza una lista dinámica donde se almacenan los objetos de tipo GesTareas.

Entre sus principales funciones se encuentran:

- **Agregar tareas**: permite registrar una nueva tarea dentro de la lista.

```
// Agregar una tarea nueva
0 referencias
public void Agregar(string titulo, string categoria, string fecha, int prioridad)
{
    GesTareas nueva = new GesTareas(siguienteId, titulo, categoria, fecha, prioridad);
    tareas.Add(nueva);
    siguienteId++;
}
```

3 Agregar tarea

- **Completar tareas**: cambia el estado de una tarea a completada.

```
// Marcar tarea como completada
0 referencias
public void Completar(int id)
{
    for (int i = 0; i < tareas.Count; i++)
    {
        if (tareas[i].Id == id)
        {
            tareas[i].Completada = true;
            return;
        }
    }
}
```

4 Completar tarea

- **Eliminar tareas**: elimina una tarea de la lista mediante su identificador.

```
// Eliminar una tarea
0 referencias
public void Eliminar(int id)
{
    for (int i = 0; i < tareas.Count; i++)
    {
        if (tareas[i].Id == id)
        {
            tareas.RemoveAt(i);
            return;
        }
    }
}
```

5 Eliminar tarea

- **Mostrar todas las tareas**: despliega todas las tareas registradas.

```
// Mostrar TODAS las tareas
0 referencias
public void MostrarTodas()
{
    Console.WriteLine("----------------------------");
    Console.WriteLine("         MIS TAREAS         ");
    Console.WriteLine("----------------------------");

    if (tareas.Count == 0)
    {
        Console.WriteLine("No hay tareas todavia");
    }
    else
    {
        foreach (GesTareas t in tareas)
        {
            Console.WriteLine(t.Mostrar());
        }
    }

    Console.WriteLine("----------------------------");
    Console.WriteLine("Tareas completadas: " + ContarCompletadas() + " / " + tareas.Count);
}
```

6 Mostrar todas las tareas

- **Mostrar tareas pendientes**: muestra únicamente aquellas que no han sido completadas.

```
// Mostrar solo pendientes
0 referencias
public void MostrarPendientes()
{
    Console.WriteLine("TAREAS PENDIENTES:");
    bool hayPendientes = false;

    foreach (GesTareas t in tareas)
    {
        if (!t.Completada)
        {
            Console.WriteLine(t.Mostrar());
            hayPendientes = true;
        }
    }

    if (!hayPendientes)
    {
        Console.WriteLine("¡No tienes tareas pendientes! ");
    }
}
```

7 Mostrar tareas pendientes

- **Mostrar tareas completadas**: muestra las tareas que ya fueron finalizadas.

```
// Mostrar solo completadas
0 referencias
public void MostrarCompletadas()
{
    Console.WriteLine("TAREAS COMPLETADAS:");
    int contador = 0;

    foreach (GesTareas t in tareas)
    {
        if (t.Completada)
        {
            Console.WriteLine(t.Mostrar());
            contador++;
        }
    }

    Console.WriteLine("Total completadas: " + contador + " de " + tareas.Count);
}
```

8 Mostrar tareas completadas

- **Buscar tareas**: permite localizar tareas mediante palabras clave.

```
// Búsqueda básica por palabra
0 referencias
public void Buscar(string palabra)
{
    Console.WriteLine("Resultados de busqueda para: \"" + palabra + "\"");
    bool encontrado = false;

    foreach (GesTareas t in tareas)
    {
        if (t.Titulo.ToLower().Contains(palabra.ToLower()) ||
            (!string.IsNullOrEmpty(t.Categoria) && t.Categoria.ToLower().Contains(palabra.ToLower())))
        {
            Console.WriteLine(t.Mostrar());
            encontrado = true;
        }
    }

    if (!encontrado)
    {
        Console.WriteLine("No se encontro nada");
    }
}
```

9 Buscar tarea

- **Contadores de tareas**: calcula el total de tareas registradas y cuántas han sido completadas.

```
// Contadores
0 referencias
public int ContarTotal()
{
    return tareas.Count;
}

1 referencia
public int ContarCompletadas()
{
    int contador = 0;
    foreach (GesTareas t in tareas)
    {
        if (t.Completada) contador++;
    }
    return contador;
}
```

10 Contadores

CONSOLA

- **Importación de la librería en la consola**: importamos la librería a la consola para pasar los datos.

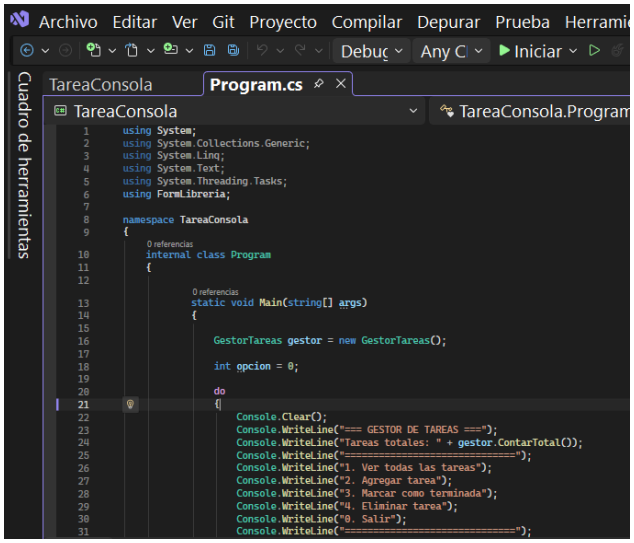

Ilustración 11 Librería

- **Programación del menú**: programamos como se verá nuestro menú en la consola al ejecutar.

```
GestorTareas gestor = new GestorTareas();

int opcion = 0;

do
{
    Console.Clear();
    Console.WriteLine("=== GESTOR DE TAREAS ===");
    Console.WriteLine("Tareas totales: " + gestor.ContarTotal());
    Console.WriteLine("============================");
    Console.WriteLine("1. Ver todas las tareas");
    Console.WriteLine("2. Agregar tarea");
    Console.WriteLine("3. Marcar como terminada");
    Console.WriteLine("4. Eliminar tarea");
    Console.WriteLine("0. Salir");
    Console.WriteLine("============================");
    Console.Write("Opción: ");

    if (!int.TryParse(Console.ReadLine(), out opcion))
    {
        opcion = -1;
    }
```

Ilustración 12 menú de la consola

- **Programación de opciones del menú**: programamos y validamos las opciones que traerá el menú de nuestra consola al ejecutar, valores válidos y valores incorrectos.

```
if (!int.TryParse(Console.ReadLine(), out opcion))
{
    opcion = -1;
}

Console.WriteLine();

if (opcion == 1)
{
    gestor.MostrarTodas();
}
else if (opcion == 2)
{
    Console.Write("Titulo de la tarea: ");
    string titulo = Console.ReadLine().Trim() ?? "";

    Console.Write("Prioridad (1 baja, 2 media, 3 alta): ");
    int prio = 2;
    if (int.TryParse(Console.ReadLine(), out int valor))
    {
        if (valor >= 1 && valor <= 3) prio = valor;
    }

    gestor.Agregar(titulo, "", "", prio);
    Console.WriteLine("Tarea agregada");
}
else if (opcion == 3)
{
```

Ilustración 13 Código del menú

```
        else if (opcion == 3)
        {
            gestor.MostrarTodas();
            Console.Write("Numero/ID de la tarea a terminar: ");
            if (int.TryParse(Console.ReadLine(), out int id))
            {
                gestor.Completar(id);
                Console.WriteLine("Marcada como terminada");
            }
            else
            {
                Console.WriteLine("Numero no valido");
            }
        }
        else if (opcion == 4)
        {
            gestor.MostrarTodas();
            Console.Write("Número/ID de la tarea a borrar: ");
            if (int.TryParse(Console.ReadLine(), out int id))
            {
                gestor.Eliminar(id);
                Console.WriteLine("Tarea borrada (si existia)");
            }
            else
            {
                Console.WriteLine("Numero no valido");
            }
        }
        else if (opcion == 0)
```

```
Consola                                    TareaConsola.Program
                }
                else if (opcion == 0)
                {
                    Console.WriteLine("saliendo....");
                }
                else
                {
                    Console.WriteLine("Opción incorrecta...");
                }

                if (opcion != 0)
                {
                    Console.WriteLine("\nPresiona ENTER para continuar...");
                    Console.ReadLine();
                }

            } while (opcion != 0);
        }
    }
}
```

- **Opción 1 del del menú**: En la opción 1 podemos ver todas las tareas que tenemos ingresadas en la consola, al agregarlas en la opción 2 se mostraran al volver a elegir la misma opción.

```
=== GESTOR DE TAREAS ===
Tareas totales: 0
==============================
1. Ver todas las tareas
2. Agregar tarea
3. Marcar como terminada
4. Eliminar tarea
0. Salir
==============================
Opción: 1

----------------------------
         MIS TAREAS
----------------------------
No hay tareas todavia
----------------------------
Tareas completadas: 0 / 0

Presiona ENTER para continuar...
```

Ilustración 14 opción 1

```
=== GESTOR DE TAREAS ===
Tareas totales: 2
==============================
1. Ver todas las tareas
2. Agregar tarea
3. Marcar como terminada
4. Eliminar tarea
0. Salir
==============================
Opción: 1

----------------------------
         MIS TAREAS
----------------------------
[ ] Reporte circuitos      Prioridad: Alta
[ ] Infografia que son las librerias      Prioridad: Media
----------------------------
Tareas completadas: 0 / 2

Presiona ENTER para continuar...
```

Ilustración 15 Opciones agregadas

- **Opción 2 del del menú**: en esta opción se agrega alguna tarea, ingresamos el nombre y prioridad de esta y se guarda en los datos de la consola.

```
=== GESTOR DE TAREAS ===
Tareas totales: 2
==============================
1. Ver todas las tareas
2. Agregar tarea
3. Marcar como terminada
4. Eliminar tarea
0. Salir
==============================
Opción: 2

Titulo de la tarea: Infografia metodos numericos
Prioridad (1 baja, 2 media, 3 alta): 1
Tarea agregada
```

Ilustración 16 Opción 2

- **Opción 3 del del menú**: al darle click a la opción 3 se despliega el menú de tareas en la cual en la parte inferior podemos seleccionar el numero de la tarea que queremos marcar como terminada, al finalizar esta mostrara una palomita de completado y el contador de tareas completadas aumentara.

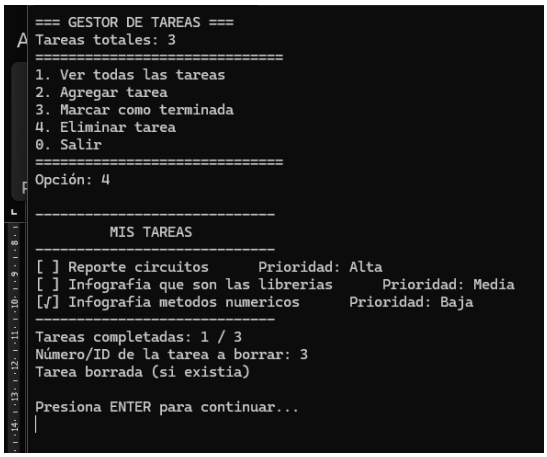

Ilustración 17 opción 3

- **Opción 4 del del menú**: Volvemos a ver le menú de las tareas y seleccionamos el numero de la tarea que deseamos eliminar del menú, al volver a ejecutar la tarea ya no se mostrara.

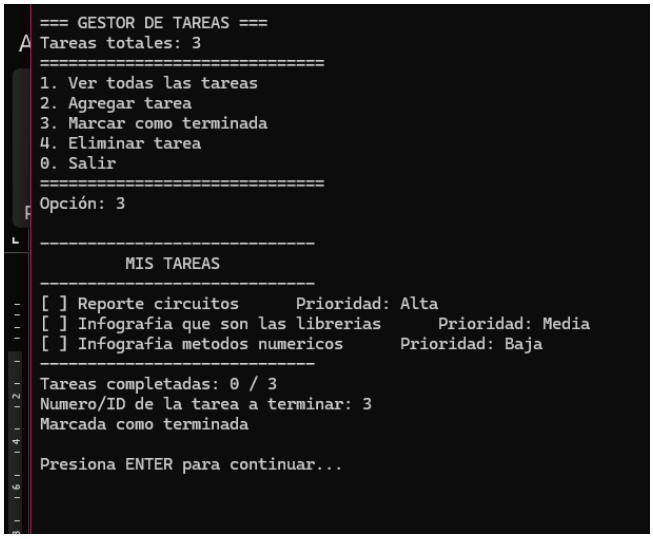

Ilustración 18 opción 4

```
=== GESTOR DE TAREAS ===
Tareas totales: 2
============================
1. Ver todas las tareas
2. Agregar tarea
3. Marcar como terminada
4. Eliminar tarea
0. Salir
============================
Opción: 1

----------------------------
         MIS TAREAS
----------------------------
[ ] Reporte circuitos      Prioridad: Alta
[ ] Infografía que son las librerias      Prioridad: Media
----------------------------
Tareas completadas: 0 / 2

Presiona ENTER para continuar...
```

Ilustración 19 Ejecución

- **Opción 0 del del menú**: en esta opción nos manda a la salida directa del programa de la consola.

```
=== GESTOR DE TAREAS ===
Tareas totales: 2
============================
1. Ver todas las tareas
2. Agregar tarea
3. Marcar como terminada
4. Eliminar tarea
0. Salir
============================
Opción: 0

saliendo....

C:\Users\gonza\source\repos\compartir\compartir k\TareaConsola\TareaConsola\bin\Debug\TareaConsola.exe (proceso 17892) s
e cerró con el código 0 (0x0).
Para cerrar automáticamente la consola cuando se detiene la depuración, habilite Herramientas ->Opciones ->Depuración ->
Cerrar la consola automáticamente al detenerse la depuración.
Presione cualquier tecla para cerrar esta ventana. . .
```

Ilustración 20 salida de la consola

FUNCIONAMIENTO GENERAL

El funcionamiento de la librería se basa en el uso de una lista dinámica que almacena objetos de tipo GesTareas. Cada vez que se registra una nueva tarea, se crea un objeto con la información ingresada y se agrega a la lista.

Posteriormente, los métodos del gestor permiten recorrer la lista para realizar diferentes operaciones, como buscar tareas, eliminarlas o marcar su estado como completado.

Este enfoque permite administrar múltiples tareas de forma sencilla y eficiente dentro del programa.

RESULTADOS

Como resultado de la práctica, se obtuvo una librería funcional que permite gestionar tareas de manera organizada. La implementación de las clases y métodos permite separar la lógica del sistema de la interfaz gráfica, facilitando la integración con aplicaciones de escritorio.
La librería proporciona una base sólida para desarrollar sistemas de organización de actividades, permitiendo ampliar sus funcionalidades en el futuro.

CONCLUSIONES

El desarrollo de esta librería permitió aplicar conceptos de programación orientada a objetos, organización modular del código y manejo de colecciones dinámicas en C#. La implementación de clases especializadas para representar tareas y administrar su funcionamiento contribuye a mantener el sistema estructurado y fácil de mantener.

Además, el uso de una librería independiente permite reutilizar la lógica del gestor de tareas en diferentes proyectos, demostrando la importancia de dividir los sistemas en componentes que puedan ser reutilizados y adaptados según las necesidades del programa.

Así también, el desarrollo de la consola de la aplicación de gestor de tareas ha sido de gran ayuda a nuestro proyecto, permitiendo una gestión eficiente y organizada de las actividades diarias como ejemplo a un problema de la vida real. La implementación de esta herramienta ha facilitado el seguimiento y cumplimiento de objetivos, contribuyendo a mejorar la productividad del usuario para un gestor de tareas.